

1. Proposed Methodology

In this study, we use 3D-CNNs for the classification of hyperspectral images of wheat seeds. Unlike 2D-CNNs, where each channel is 2-dimensional and the convolutions happen in only spatial directions, 3D-CNNs operate on volumetric data, where each channel is 3-dimensional, and the convolutions occur in both spatial and spectral directions. This property of 3D-CNNs makes them suitable for extracting nice features from the hyperspectral images. To leverage this advantage, we develop three distinct 3D-CNN architectures by extending the baseline 2D architectures of ResNet-34, DenseNet-121, and ConvNeXt-T. This section describes the 3D models and their training process in detail.

3a. 3D-ResNet Model

The first model, 3D-ResNet, was constructed by changing the first convolution layer and the number of filters in subsequent convolution layers of the original ResNet-34 model. The first convolution layer was redesigned to preserve the spatial dimensions of the input image while reducing the spectral dimension by half. This modification was necessary because the input images are relatively small, and reducing the spatial dimensions at this stage would result in excessively small feature maps in the deeper layers. Next, since 3D-CNNs consider the entire 3D hyperspectral image as a single channel, we require a smaller number of filters in subsequent convolution layers. To account for this, we reduced the number of filters in these layers to one-fourth of their original values. Additionally, the final fully connected layer was adjusted to contain 50 neurons instead of 1,000, corresponding to the number of target classes.

To evaluate performance, we compared the 3D-ResNet model with its 2D counterpart. Similar modifications were applied to the 2D-ResNet-34 model, referred to as the "Modified 2D-ResNet-34." The complete architectures of all models are presented in Table 1, with key modifications highlighted in bold. Furthermore, we calculated the number of parameters and Floating-Point Operations (FLOPs). The 3D-ResNet model demonstrated a significant reduction in parameters compared to the original 2D-ResNet-34 architecture. Additionally, the modified 2D model had approximately one-third the number of parameters and one-tenth the FLOPs of its 3D counterpart.

ResNet-34	Original 2D-ResNet-34	Output size of Original 2D-ResNet-34	Modified 2D-ResNet-34	Output size of Modified 2D-ResNet-34	3D-ResNet	Output size of 3D-ResNet
Input		224 x 224 x 3		56 x 56 x 147		56 x 56 x 147 x 1
Conv1	7x7, 64, stride 2, padding 3	112 x 112 x 64	3x3, 64, stride 1, padding 1	56 x 56 x 64	3x3x3, 64, stride (2,1,1), padding 1	56 x 56 x 74 x 64
Max Pool	3x3, stride 2, padding 1	56 x 56 x 64	3x3, stride 2, padding 1	28 x 28 x 64	3x3x3, stride 2, padding 1	28 x 28 x 37 x 64
Conv2	[3x3, 64, s1 3x3, 64, s1] x 3	56 x 56 x 64	[3x3, 16 , s1 3x3, 16 , s1] x 3	28 x 28 x 16	[3x3x3, 16 , s1 3x3x3, 16 , s1] x 3	28 x 28 x 37 x 16
Conv3	[3x3, 128, s1 3x3, 128, s1] x 4	28 x 28 x 128	[3x3, 32 , s2 3x3, 32 , s1] x 4	14 x 14 x 32	[3x3x3, 32 , s1 3x3x3, 32 , s1] x 4	14 x 14 x 19 x 32

Conv4	[3×3, 256, s1 3×3, 256, s1] x 6	14 x 14 x 256	[3×3, 64 , s2 3×3, 64 , s1] x 6	7 x 7 x 64	[3×3×3, 64 , s1 3×3×3, 64 , s1] x 6	7 x 7 x 10 x 64
Conv5	[3×3, 512, s1 3×3, 512, s1] x 3	7 x 7 x 512	[3×3, 128 , s2 3×3, 128 , s1] x 3	4 x 4 x 128	[3×3×3, 128 , s1 3×3×3, 128 , s1] x 3	4 x 4 x 5 x 128
Classifier	average pool	1 x 1 x 512	average pool	1 x 1 x 128	average pool	1 x 1 x 1 x 128
	flatten	512	flatten	128	flatten	128
	linear	1000	linear	50	linear	50
Parameters		21,797,672		1,432,082		3,998,738
FLOPs		3,680,019,432		332,183,730		3,935,935,282

Table 1: The architecture of the original 2D-ResNet-34 model, Modified 2D-ResNet-34 model and 3D-ResNet model along with their respective Output sizes. (Main changes are shown in bold letters, FLOPs - Floating Point Operations, s - stride)

3b. 3D-DenseNet Model

The next architecture, 3D-DenseNet, was derived from the original 2D-DenseNet-121 model by modifying the initial convolutional layer and adjusting the growth rate (k) parameter. The initial convolutional layer was again modified to prevent excessive reduction of spatial dimensions in the deeper layers. In DenseNet, the number of channels is regulated by the growth rate parameter within each dense block. Therefore, in the 3D model, we reduced this value from 32 to 6 to accommodate the increased computational complexity of 3D convolutions and the higher memory requirements of large feature maps. Similarly, an equivalent 2D architecture was designed with appropriate modifications for comparative analysis. The complete architecture is presented in Table 2. In the 3D-DenseNet model, the number of parameters is reduced by nearly 18 times compared to the original 2D-DenseNet-121 model. The modified 2D model has a similar number of parameters, but its FLOPs are reduced significantly as compared to the 3D architecture.

DenseNet-121	Original 2D-DenseNet-121 (k=32)	Output size of Original 2D-DenseNet-121	Modified 2D-DenseNet-121 (k=6)	Output size of Modified 2D-DenseNet-121	3D-DenseNet (k=6)	Output size of 3D-DenseNet-121
Input		224 x 224 x 3		56 x 56 x 147		56 x 56 x 147 x 1
Conv1	7×7, 64, stride 2, padding 3	112 x 112 x 64	3×3, 64, stride 1, padding 1	56 x 56 x 64	3×3×3, 64, stride (2,1,1), padding 1	56 x 56 x 74 x 64
Max Pool	2x2, stride 2, padding 0	56 x 56 x 64	2x2, stride 2, padding 0	28 x 28 x 64	2x2x2, stride 2, padding 0	28 x 28 x 37 x 64
Dense Block 1	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 6	56 x 56 x 256	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 6	28 x 28 x 100	[1×1x1, 4k, s1, p0 3×3x3, k, s1, p1] x 6	28 x 28 x 37 x 100
Transition Layer 1	conv(1x1,128,s1)	28 x 28 x 128	conv(1x1,128,s1)	14 x 14 x 50	conv(1x1x1,50,s1)	14 x 14 x 18 x 50
	maxpool(2x2,s2)		maxpool(2x2,s2)		maxpool(2x2x2,s2)	

Dense Block 2	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 12	28 x 28 x 512	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 12	14 x 14 x 122	[1×1x1, 4k, s1, p0 3×3x3, k, s1, p1] x 12	14 x 14 x 18 x 122
Transition Layer 2	conv(1x1,256,s1)	14 x 14 x 256	conv(1x1,256,s1)	7 x 7 x 61	conv(1x1x1,61,s1)	7 x 7 x 9 x 61
	maxpool(2x2,s2)		maxpool(2x2,s2)		maxpool(2x2x2,s2)	
Dense Block 3	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 24	14 x 14 x 1024	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 24	7 x 7 x 205	[1×1x1, 4k, s1, p0 3×3x3, k, s1, p1] x 24	7 x 7 x 9 x 205
Transition Layer 3	conv(1x1,512,s1)	7 x 7 x 512	conv(1x1,512,s1)	3 x 3 x 102	conv(1x1x1,102,s1)	3 x 3 x 4 x 102
	maxpool(2x2,s2)		maxpool(2x2,s2)		maxpool(2x2x2,s2)	
Dense Block 4	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 16	7 x 7 x 1024	[1×1, 4k, s1, p0 3×3, k, s1, p1] x 16	3 x 3 x 198	[1×1x1, 4k, s1, p0 3×3x3, k, s1, p1] x 16	3 x 3 x 4 x 198
Classifier	average pool	1 x 1 x 1024	average pool	1 x 1 x 198	average pool	1 x 1 x 1 x 198
	flatten	1024	flatten	198	flatten	198
	linear	1000	linear	50	linear	50
Parameters		7,978,856		387,796		455,188
FLOPs		2,899,843,048		305,900,534		2,150,821,018

Table 2: The architecture of the 2D-DenseNet-121, Modified 2D-DenseNet-121 and 3D-DenseNet model along with their respective Output sizes (Main changes are shown in bold letters, FLOPs - Floating Point Operations, s - stride, p - padding)

3c. 3D-ConvNext Model

The ConvNeXt-T architecture was originally developed as an improved variant of ResNet-50, incorporating eleven key modifications that enhanced its performance on image classification tasks. The 3D-ConvNeXt model extends the 2D-ConvNeXt-T architecture with appropriate modifications. Specifically, the initial convolutional layer and the number of filters in subsequent layers were adjusted similarly to the 3D-ResNet model. Additionally, we modified the kernel size, stride, and padding in the down sampling layers to ensure that the spatial dimensions remained consistent with those in the 3D-ResNet model.

To facilitate comparative analysis, we also designed an equivalent 2D ConvNeXt-T model with corresponding modifications. The complete architectures of all models are presented in Table 3. In the 3D model, the number of parameters is reduced to approximately one-tenth, while the FLOPs are nearly halved compared to the original architecture. The modified 2D model exhibits a significant reduction in floating-point operations compared to its 3D counterpart.

ConvNeXt-T	Original 2D-ConvNeXt-T	Output size of original 2D-ConvNeXt-T	Modified 2D-ConvNeXt-T	Output size of modified 2D-ConvNeXt-T	3D-ConvNeXt	Output size of 3D-ConvNeXt-T
input		224 x 224 x 3		56 x 56 x 147		56 x 56 x 147 x 1
stem	4×4, 96, stride 4, padding 0	56 x 56 x 96	4×4, 24, stride 2, padding 1	28 x 28 x 24	4x4x4, 24, stride (4,2,2), padding 1	28 x 28 x 37 x 24

res2	[d7×7, 96 1×1, 384 1×1, 96] x 3	56 x 56 x 96	[d7×7, 24 1×1, 96 1×1, 24] x 3	28 x 28 x 24	[d7×7×7, 24 1×1×1, 96 1×1×1, 24] x 3	28 x 28 x 37 x 24
ds3	2x2, 192, stride 2, padding 0		3x3, 48, stride 2, padding 1		3x3x3, 48, stride 2, padding 1	
res3	[d7×7, 192 1×1, 768 1×1, 192] x 3	28 x 28 x 192	[d7×7, 48 1×1, 192 1×1, 48] x 3	14 x 14 x 48	[d7×7×7, 48 1×1×1, 192 1×1×1, 48] x 3	14 x 14 x 19 x 48
ds4	2x2, 384, stride 2, padding 0		3x3, 96, stride 2, padding 1		3x3x3, 96, stride 2, padding 1	
res4	[d7×7, 384 1×1, 1536 1×1, 384] x 9	14 x 14 x 384	[d7×7, 96 1×1, 384 1×1, 96] x 9	7 x 7 x 96	[d7×7×7, 96 1×1×1, 384 1×1×1, 96] x 9	7 x 7 x 10 x 96
ds5	2x2, 768, stride 2, padding 0		3x3, 192, stride 2, padding 1		3x3x3, 192, stride 2, padding 1	
res5	[d7×7, 768 1×1, 3072 1×1, 768] x 3	7 x 7 x 768	[d7×7, 192 1×1, 768 1×1, 192] x 3	4 x 4 x 192	[d7×7×7, 192 1×1×1, 768 1×1×1, 192] x 3	4 x 4 x 5 x 192
classifier	average pool	1 x 1 x 768	average pool	1 x 1 x 192	average pool	1 x 1 x 1 x 192
	flatten	768	flatten	192	flatten	192
	linear	1000	linear	50	linear	50
Parameters		28,582,504		1,996,754		2,864,162
FLOPs		4,488,347,368		128,329,586		2,379,070,706

Table 3: The architecture of the 2D-ConvNext-T, Modified 2D-ConvNext-T and 3D-ConvNext-T model along with their respective output sizes. (Main changes are shown in bold letters, FLOPs - Floating Point Operations, d - depth wise convolution, res - residual block, ds - down sampling layer)

3d. Models Training

To ensure a straightforward and consistent training process, we retained settings closely aligned with standard configurations. Both 2D and 3D models were trained using the parameters outlined in Table 4. A batch size of 64 was used for all models, except DenseNet, which required a smaller batch size due to its higher memory demands. The ResNet and DenseNet models were optimized using stochastic gradient descent (SGD) with Nesterov momentum (0.9), while ConvNeXt was trained using the AdamW optimizer. To further enhance training efficiency, different learning rate scheduling strategies were adopted for each model. For ResNet, the ReduceLROnPlateau scheduler was applied with a reduction factor of 0.1 and a patience of 5 epochs, allowing adaptive adjustment based on validation performance. DenseNet employed a StepLR scheduler, reducing the learning rate by half every 10 epochs. For ConvNeXt, training began with a 5-epoch linear warm-up, followed by a CosineAnnealingLR schedule to gradually decay the learning rate over time. Learning rates and weight decay parameters were fine-tuned to maximize model performance within the fewest epochs. All models were trained for up to 100 epochs, with early stopping implemented if no improvement in validation accuracy was observed for 10 consecutive epochs.

Model Name	Batch Size	Optimizer	Initial learning rate	LR Scheduler	Weight decay	Early Stopping
2D/3D ResNet	64	SGD	0.001	ReduceLROnPlateau	0.01	yes
2D/3D DenseNet	32	SGD	0.0005	StepLR	0.025	yes
2D/3D ConvNeXt	64	AdamW	0.0005	CosineAnnealingLR	0.05	yes

Table 4: The parameters used for the training of 2D and 3D CNN models